

TP3 – Détection d’objets 3D

Table des matières

1	Introduction	3
2	Exercice 1 : Configuration des communications ROS 2	3
2.1	Objectif	3
2.2	Ce que vous devez remplir	3
2.3	Indices	3
3	Exercice 2 : Extraction de la matrice intrinsèque	4
3.1	Objectif	4
3.2	Rappel théorique	4
3.3	Ce que vous devez remplir	4
3.4	Vérification	4
4	Exercice 3 : Conversion de l’unité de profondeur	4
4.1	Objectif	4
4.2	Rappel théorique	5
4.3	Ce que vous devez remplir	5
5	Exercice 4 : Modèle sténopé (Projection inverse)	5
5.1	Objectif	5
5.2	Rappel théorique	5
5.3	Ce que vous devez remplir	5
6	Exercice 5 : Calcul des dimensions physiques	5
6.1	Objectif	5
6.2	Rappel théorique	5
6.3	Ce que vous devez remplir	6
7	Exercice 6 : Publication du message ROS 2	6
7.1	Objectif	6
7.2	Rappel théorique	6
7.3	Ce que vous devez remplir	6
8	Mise en place de l’environnement et test	6
8.1	Transfert du modèle sur le robot (Git clone)	6
8.2	Export du modèle en format OpenVINO	7
8.3	Modifications à apporter au code	7
8.4	Ajout de la fonction detecter_couleur_feu	8

8.5	Création du custom topic	8
8.6	Compilation et lancement	8

1 Introduction

Dans ce TP, vous allez compléter progressivement un squelette de code de détection 3D en ROS2. Pour faire ce TP vous devez avoir fait celui d'avant. Le fichier `detector_3d.py` vous est fourni. Chaque exercice correspond à une partie théorique vue précédemment et vous indique précisément ce que vous devez remplir. Vous allez comprendre et mettre en place de la détection 3D avec une caméra de profondeur. Tout ça sera mis en place dans l'environnement que nous avons créé dans le tp précédent. Il suffit juste de créer un `.py` et le mettre dans le package de détection (tout en ajoutant les bonnes dépendances)

Pour l'IA vous utiliserez le modèle que vous avez fait avec Roboflow. Le but final sera de faire la même chose que dans le TP de détection 2D avec noVNC sauf qu'on aura maintenant l'objet en 3D ET la bonne classification d'images (feux de circulation, panneaux, voiture, etc)

Finalement, nous créerons un topic qui publiera toutes les valeurs qui nous intéressent et que mes camarades pourront utiliser dans leurs codes.

2 Exercice 1 : Configuration des communications ROS 2

2.1 Objectif

Configurer les subscribers et publishers pour récupérer les données de la caméra RealSense et publier les résultats de détection.

2.2 Ce que vous devez remplir

Dans la méthode `__init__`, trouvez les zones # À COMPLÉTER et remplissez :

1. **Le subscriber `sub_info`** : le type de message et le nom du topic de calibration.
2. **Le subscriber `sub_rgb`** : le type de message et le nom du topic image couleur.
3. **Le subscriber `sub_depth`** : le type de message et le nom du topic image de profondeur.
4. **Le publisher `pub_image`** : le type de message pour publier une image annotée.
5. **Le publisher `pub_points`** : le type de message pour publier des coordonnées 3D.

2.3 Indices

Pour trouver les noms des topics disponibles :

```
ros2 topic list | grep camera
```

Pour voir le type d'un message :

```
ros2 interface show # votre message
```

Les types de messages dont vous aurez besoin sont déjà importés en haut du fichier.

3 Exercice 2 : Extraction de la matrice intrinsèque

3.1 Objectif

Dans la méthode `callback_camera_info`, extraire les paramètres f_x , f_y , c_x et c_y du message `CameraInfo` reçu.

3.2 Rappel théorique

La matrice intrinsèque K est structurée comme suit :

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (1)$$

Dans le message ROS `CameraInfo`, ces valeurs sont stockées dans un tableau linéaire `k` de 9 éléments :

Elle se présente sous une forme 3×3 :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y : Les distances focales exprimées en pixels. Elles décrivent le grossissement de la lentille sur les axes X et Y.
- c_x, c_y : Les coordonnées en pixels du **centre optique** (l'endroit où l'axe de la lentille traverse le capteur, proche du centre de l'image).

Dans ROS 2, pour simplifier le transfert réseau, cette matrice est aplatie dans un tableau à une seule dimension de 9 éléments : `[fx, 0, cx, 0, fy, cy, 0, 0, 1]`.

FIGURE 1 – Structure du tableau `k` dans le message `CameraInfo`

3.3 Ce que vous devez remplir

Dans `callback_camera_info`, remplissez les quatre variables `self.fx`, `self.fy`, `self.cx`, `self.cy` en lisant les bons indices du tableau `msg.k`.

3.4 Vérification

```
ros2 topic echo /camera/camera/color/camera_info | grep -A 10 "k:"
```

4 Exercice 3 : Conversion de l'unité de profondeur

4.1 Objectif

Dans la méthode `pixel_to_3d`, calculer la profondeur Z en mètres à partir du patch de pixels de profondeur.

4.2 Rappel théorique

La RealSense D435 fournit la profondeur en **millimètres** au format `uint16`. Pour convertir en mètres, il faut diviser par 1000. La valeur de la profondeur est fournie en `uint16` car `uint8` ne permettrait d'aller que jusqu'à 25.5cm de distance (valeur de 0 à 255 mm) alors que `uint16` permet d'aller jusqu'à 65 535 mètres (valeurs de 0 à 65 535 mm).

Pour plus de robustesse face au bruit du capteur, on utilise la **médiane** plutôt que la moyenne sur la fenêtre de 10×10 pixels déjà extraite dans le code.

4.3 Ce que vous devez remplir

Calculez Z en une ligne : prenez la médiane du tableau `valid` et convertissez en mètres.

5 Exercice 4 : Modèle sténopé (Projection inverse)

5.1 Objectif

Toujours dans `pixel_to_3d`, calculer les coordonnées spatiales X et Y en mètres à partir du pixel (u, v) et de la profondeur Z .

5.2 Rappel théorique

Le modèle sténopé relie l'espace 3D à l'image 2D. La projection inverse donne :

$$X = \frac{(u - c_x) \cdot Z}{f_x} \quad Y = \frac{(v - c_y) \cdot Z}{f_y} \quad (2)$$

5.3 Ce que vous devez remplir

Calculez X et Y en appliquant les formules ci-dessus avec `self.fx`, `self.fy`, `self.cx`, `self.cy`.

6 Exercice 5 : Calcul des dimensions physiques

6.1 Objectif

Dans la méthode `taille_reelle`, calculer la largeur et la hauteur réelles de l'objet détecté en mètres.

6.2 Rappel théorique

On applique le théorème de Thalès à l'optique. La taille réelle d'un objet se calcule depuis sa boîte englobante en pixels :

$$W = \frac{(x_2 - x_1) \cdot Z}{f_x} \quad H = \frac{(y_2 - y_1) \cdot Z}{f_y} \quad (3)$$

6.3 Ce que vous devez remplir

Calculez `largeur_m` et `hauteur_m` en appliquant les formules ci-dessus. Les coordonnées (x_1, y_1, x_2, y_2) de la boîte YOLO sont passées en paramètre de la méthode.

7 Exercice 6 : Publication du message ROS 2

7.1 Objectif

Dans `callback_rgbd`, remplir et publier le message `PointStamped` avec les coordonnées 3D calculées.

7.2 Rappel théorique

Un message `PointStamped` contient deux champs :

- `header` : le timestamp et le frame de référence (toujours à copier depuis le message reçu)
- `point` : les coordonnées `x`, `y`, `z` en mètres

7.3 Ce que vous devez remplir

Dans la boucle de détection, remplissez l'objet `pt` de type `PointStamped` avec les valeurs `X`, `Y`, `Z` calculées, puis publiez-le.

8 Mise en place de l'environnement et test

Vous avez normalement maintenant un code capable de détecter des objets en 3D. Nous allons maintenant mettre en place le modèle que vous avez entraîné et vous allez pouvoir tester le programme.

8.1 Transfert du modèle sur le robot (Git clone)

Le fichier `best.pt` contenant les poids du modèle entraîné se trouve sur le Github. Il faut le transférer vers l'hôte du robot via un Git clone, en utilisant la commande :

Depuis un terminal de robot, dans le docker et dans le bon dossier `models`, exécutez :

```
git clone <l'url GitHub>
```

Une fois la commande terminée, le fichier `best.pt` doit se trouver sur l'hôte, dans :

```
~/roskit_ws/src/detection_pkg/models/best.pt
```

8.2 Export du modèle en format OpenVINO

Comme pour le modèle `yolo11n.pt` du TP de détection 2D, le modèle `best.pt` doit être converti au format OpenVINO pour pouvoir tourner efficacement sur le GPU intégré Intel.

Dans le terminal du **Docker**, placez-vous dans le dossier `models/` et exportez le modèle :

```
cd ~/roskit_ws/src/detection_pkg/models

python3 -c "
from ultralytics import YOLO
model = YOLO('best.pt')
model.export(format='openvino', half=True)
print('Export OpenVINO terminé !')
"
```

Cette commande génère un dossier `best_openvino_model/` contenant notamment le fichier `best.xml`, exactement comme `yolo11n_openvino_model/` contenait `yolo11n.xml` dans le TP précédent.

8.3 Modifications à apporter au code

Le squelette `detector_3d.py` que vous avez complété dans la partie précédente reste valable. Les seules modifications nécessaires sont les suivantes :

1. **Chemin du modèle** : le paramètre `model_path` (ou la valeur par défaut déclarée dans `declare_parameter`) doit pointer vers le nouveau dossier `best_openvino_model` au lieu de `yolo11n_openvino_model`.
2. **Nom du fichier XML** : dans la méthode `__init__`, la ligne qui construit le chemin vers le modèle :

```
model_xml = os.path.join(model_path, 'yolo11n.xml')
```

doit devenir :

```
model_xml = os.path.join(model_path, 'best.xml')
```

3. **Liste `self.class_names`** : remplacez la liste des 80 classes COCO par la liste des classes de votre nouveau modèle, dans le bon ordre (voir le fichier `data.yaml` fourni avec `best.pt`).
4. **`setup.py`** : deux points à vérifier dans le fichier `setup.py` du package `detection_pkg` :
 - le bloc `data_files` qui copie les modèles dans l'installation doit pointer vers le nouveau dossier `models/best_openvino_model/*` (et non plus `yolo11n_openvino_model`);
 - dans `entry_points`, l'entrée `console_scripts` doit associer la commande `detector_3d` à votre script, par exemple :

```
'detector_3d = detection_pkg.detector_3d:main',
```

Aucune autre partie du code (calcul de X , Y , Z , taille réelle, publication du `PointStamped`) n'a besoin d'être modifiée : ces calculs sont indépendants du modèle utilisé, ils ne dépendent que de la boîte englobante renvoyée par YOLO et de l'image de profondeur.

8.4 Ajout de la fonction `detecter_couleur_feu`

Comme vu dans le TP précédent, nous ne pouvons pas entraîner le modèle avec une seule classe de feux (vert, orange et rouge) car le modèle ne comprend pas ce qu'il doit apprendre. Ainsi nous avons séparé l'entraînement en trois classes. Mais vous verrez que si vous laissez la détection se faire selon les classes Yolo le modèle se trompera souvent. Le modèle ne peut pas être correctement analyser la couleur du feu mais il sait reconnaître parfaitement si c'est un feu ou non. Donc dans le processing de l'image nous allons regrouper les trois classes feux en une seule, et c'est une analyse cv2 plus détaillée et précise qui renverra la couleur. C'est à ça que sert la fonction `detecter_couleur_feu`.

8.5 Création du custom topic

Nous allons finalement analyser le custom topic du code et allons le mettre en place et l'étudier. Avec la ligne :

```
self.pub_meta = self.create_publisher(Detection2DArray,  
'/detection/metadata', 10)
```

Vous publiez sur un topic nommé `/detection/metadata` et de type `Detection2DArray`. Pour ce topic on utilisera une bibliothèque ros2 déjà existante qui est `vision_msgs`. Pour cela vous devez l'installer premièrement dans votre docker :

```
sudo apt update  
sudo apt install ros-humble-vision-msgs
```

Et finalement vous devez rajouter dans le `package.xml`

```
<depend>vision_msgs</depend>
```

Maintenant faites un :

```
ros2 topic echo /detection/metadata --once
```

Qu'observez-vous ? Comment est organisé le topic ? Quelles informations peut on trouver ?

8.6 Compilation et lancement

La procédure est identique à celle du TP de détection 2D :

```
source /opt/ros/humble/setup.bash  
cm  
source $ABSOLUTE_WORKSPACE/install/setup.bash  
ros2 run detection_pkg detector_3d --ros-args -p device:=GPU
```

Vous pouvez ensuite, comme précédemment, visualiser le résultat via noVNC en ajoutant le topic `/detection/image_3d` dans rviz2 (add -> by topic).